



Republic of Tunisia
Ministry of High Education and Scientific Research

University of Monastir
High Institute of Computer Science and Mathematics of Monastir
Department of Technology



Graduation Project

In order to obtain the
National Diploma of Engineering in Applied Sciences and Technology

Major:
Electronics: Microelectronics

Presented by:

Najd Elaoud and Selim Triki

**Design and development of the IOBOX-BWS industrial platform
for data acquisition, processing, and control**

Presented X July 2026 in front of the jury members :

Mrs. UNKNOWN	President
Mr. UNKNOWN	Lecturer
Mr. Abdessalem Ben Abdelali	Academic Supervisor
Mr. Jamel Hmidi	Industrial Supervisor

Academic year: 2025/2026

ACKNOWLEDGEMENTS

I am humbled and grateful to all who have helped me translate these ideas into something concrete that goes far beyond a simple level.

I would like to express my sincere gratitude to my academic supervisor, **Mr. Abdessalem BEN ABDELALI**, for his guidance throughout this project, for reviewing my work, and for his valuable remarks and suggestions that helped improve the quality of this report.

I would also like to express my deepest appreciation to my industrial supervisor, **Mr. Jamel HMIDI**, for his continuous support, technical guidance, and availability throughout the internship. His expertise, advice, and encouragement were invaluable to the successful completion of this project.

Last but not least, I am deeply grateful to all the jury members **Mr. UNKNOWN** and **Mrs. UNKNOWN** for agreeing to evaluate this work.

Without your support, this accomplishment would not have been achieved. I really appreciate it.

DEDICATION

To my precious family, I would never be where I am now without you.

*To my dear father **Noureddine**:*

Thank you for your hard work, guidance, and the values you have instilled in me throughout my life. Your sacrifices and dedication have always inspired me to strive for success. May God bless you with health, happiness, and a long life.

*To my dear mother **Soufia**:*

You have done more than a mother can do to ensure that her children follow the right path in their lives and studies. I dedicate this work to you as a testimony of my deep love. May God, the Almighty, preserve you and grant you health, long life, and happiness.

*To my dear sister **Nibras**:*

Thank you for your constant support, encouragement, and affection. Having you by my side has always been a source of strength and motivation.

To my dear Friends, Words can't express how grateful I am to have your friendship in my life. Thank you for supporting me, accepting me, and being wonderful friends.

Lastly, to all the kind people I met during my six years here, to all the institute students, professors, administrators and workers, thank you for being part of my exciting university career.

Najd Elaoud

DEDICATION

*“What you leave behind is not what is engraved in stone monuments, but what is woven into the lives of others.” — **Pericles***

*To my dear father **Lotfi**:*

This project and my success in my academic journey could not have been achieved without your presence in my life. I am deeply grateful for the comfort and opportunities you provided for us, and for teaching me that with dedication and perseverance nothing is impossible. I hope that one day I can repay you, even in a small way, for all that you have done.

*To my dear mother **Monia**:*

The love you gave us made us stronger, and the dedication and support you offered whenever we fell down or felt low is the most precious gift in life. Thank you for being the light that guided us and for showing us the strength of unconditional love.

*To my dear sister **Ahlem**:*

Thank you for bringing happiness into my life and for being my partner through both struggles and hard work. Your emotional support has been invaluable, and I am truly grateful for the strength and companionship you have given me.

*To my dear brother **Youssef**:*

I cannot wait to see you thrive and shine in life, overcoming every turbulence along the way. I will always be here as your mentor whenever you need guidance.

*To my dear **friends**, Thank you for all the support you have given me and for the joy you bring into my life. I am truly grateful that you accept every side of my personality, and I cherish the real human connection we share. Seeing you succeed in life is part of my own dreams, and I look forward to celebrating your achievements alongside you.*

SELIM TRIKI

ABSTRACT

The development of flexible and interoperable industrial platforms is essential for addressing the growing integration challenges of Industrial Internet of Things (IIoT) systems. This report presents the design and development of the IOBOX, a configurable industrial Input/Output platform intended to simplify the acquisition, processing, control, and exchange of data between sensors, actuators, and supervisory systems.

The developed platform is based on an STM32 microcontroller and integrates multiple industrial communication interfaces, peripheral drivers, and sensor management modules within a modular software architecture. The implementation emphasizes hardware abstraction, software reusability, and scalability, enabling efficient integration of heterogeneous devices while reducing development effort.

The resulting solution provides a versatile and reusable platform for industrial monitoring, automation, and IoT applications. By improving interoperability and simplifying system integration, the IOBOX contributes to the development of more efficient and scalable industrial solutions.

Keywords: Industrial IoT, Embedded Systems, STM32, IO-Box, Data Acquisition, Industrial Communication Protocols, Automation, Software Architecture.

Le développement de plateformes industrielles flexibles et interopérables est devenu essentiel pour répondre aux défis croissants d'intégration des systèmes de l'Internet Industriel des Objets (IIoT). Ce rapport présente la conception et le développement de l'IOBOX, une plateforme industrielle d'Entrées/Sorties configurable destinée à simplifier l'acquisition, le traitement, le contrôle et l'échange de données entre les capteurs, les actionneurs et les systèmes de supervision.

La plateforme développée repose sur un microcontrôleur STM32 et intègre plusieurs interfaces de communication industrielle, pilotes de périphériques et modules de gestion de capteurs au sein d'une architecture logicielle modulaire. L'implémentation met l'accent sur l'abstraction matérielle, la réutilisabilité logicielle et l'évolutivité, permettant une intégration efficace de dispositifs hétérogènes tout en réduisant les efforts de développement.

La solution obtenue constitue une plateforme polyvalente et réutilisable pour les applications de supervision industrielle, d'automatisation et d'IoT. En améliorant l'interopérabilité et en simplifiant l'intégration des systèmes, l'IOBOX contribue au développement de solutions industrielles plus efficaces et évolutives.

Mots-clés : Internet Industriel des Objets (IIoT), Systèmes Embarqués, STM32, IOBOX, Acquisition de Données, Protocoles de Communication Industrielle, Automatisation.

List of Figures	vi
List of Tables	vi
Glossary of Acronyms	vi
General introduction	1
1 General Project Context	3
1.1 Introduction	3
1.2 Academic Institution	3
1.3 Host Company Presentation	4
1.3.1 Company Overview	4
1.3.2 Organization Chart	4
1.3.3 Main Activities and Services	5
1.4 Project Specification	7
1.4.1 Problem Statement	7
1.4.2 Presentation of the IOBOX Platform	8
1.4.3 Project Objectives	8
1.4.4 Development Methodology	9
1.5 Conclusion	11
2 General Concept and System Components	12
2.1 Introduction	12
2.2 General Concept of the IO-Box Platform	12
2.3 Hardware components	13

2.3.1	STM32G474MBTx Microcontroller	13
2.3.2	EEPROM Memory: M95M01-RDW6TP	14
2.3.3	Sensors	15
2.4	Peripherals	19
2.4.1	Digital-to-Analog Converter (DAC)	19
2.4.2	Analog to Digital Converter (ADC)	20
2.4.3	Timers	21
2.4.4	Pulse Width Modulation (PWM)	21
2.5	Communication Protocols	22
2.5.1	I2C Protocol	22
2.5.2	Serial Peripheral Interface (SPI)	23
2.5.3	UART Protocol	24
2.5.4	RS485 Communication	25
2.5.5	Modbus Protocol	26
2.5.6	Controller Area Network (CAN)	27
2.5.7	LIN Protocol	29
2.6	Software tools	29
2.6.1	Visual Studio Code	29
2.6.2	IAR Embedded Workbench	30
2.6.3	STM32CubeMX	30
2.6.4	GitLab	31
2.6.5	Conclusion	31
3	Design and Practical Implementation	33
3.1	Introduction	33
3.2	Hardware architecture	33
3.3	Software architecture	33
3.3.1	Board support package (BSP) layer	33
3.3.2	Manager layer	33
3.3.3	Application layer	33
3.4	Conclusion	33

LIST OF FIGURES

1.1	Be Wireless Solutions (BWS) logo	4
1.2	Organization chart of Be Wireless Solutions	5
1.3	Electricity consumption monitoring solution	5
1.4	Water consumption monitoring solution	6
1.5	Fleet and fuel management solution	6
1.6	Remote equipment monitoring and control	7
1.7	Cold-chain monitoring solution	7
1.8	V-Model development lifecycle	11
2.1	STM32G474 Microcontroller (physical package)	13
2.2	M95M01-RDW6TP EEPROM memory chip	15
2.3	ENS210 digital temperature and humidity sensor	16
2.4	OPT3001DNPR ambient light sensor	17
2.5	HTS221TR temperature and humidity sensor	19
2.6	Example of analog signal generated by DAC	19
2.7	Basic ADC conversion from analog input to digital output	21
2.8	Illustration of PWM signals with different duty cycles (25%, 50%, and 75%)	22
2.9	I2C frame format	23
2.10	I2C communication bus architecture	23
2.11	Basic SPI communication between master and slave devices	24
2.12	UART frame format	25
2.13	RS485 differential bus with multi-node configuration	26
2.14	Typical Modbus RTU frame structure	27
2.15	Basic CAN frame	28
2.16	Basic CAN bus communication with multiple nodes	28
2.17	LIN frame structure	29

2.18 Visual Studio Code interface	30
2.19 IAR Embedded Workbench IDE	30
2.20 STM32CubeMX configuration interface	31
2.21 GitLab logo	31

LIST OF TABLES

1.1	Application of the V-Model to the Project	10
2.1	Product Attributes — STM32G474 Microcontroller	14
2.2	Key Characteristics of M95M01-RDW6TP EEPROM	15
2.3	Key Characteristics of ENS210 Sensor	16
2.4	Key Characteristics of OPT3001DNPR Sensor	17
2.5	Key Characteristics of HTS221TR Sensor	18

GLOSSARY OF ACRONYMS

BSP: Board Support Package

UART : Universal Asynchronous Receiver-Transmitter

LIN: Local Interconnect Network

USB : Universal Serial Bus

CAN : Controller Area Network

I2C : Inter-Integrated Circuit

SPI : Serial Peripheral Interface

DAC : Digital to Analog Converter

ADC : Analog to Digital Converter

GPIO : General Purpose Input Output

PID : Proportional Integral Derivative

GENERAL INTRODUCTION

Industrial systems are increasingly relying on connected devices capable of acquiring, processing, and exchanging information in real time. In such environments, sensors, actuators, and supervisory systems must interact efficiently through various communication interfaces and protocols. However, integrating these heterogeneous components often requires significant development effort, particularly when flexibility, scalability, and interoperability are required.

To address these challenges, Be Wireless Solutions (BWS) launched the development of the IOBox platform, a configurable industrial Input/Output system designed to serve as a universal interface between field devices and monitoring or control systems. The platform is intended to simplify the integration of sensors and actuators while providing data acquisition, signal processing, control, and communication functionalities within a single solution.

The IOBox supports multiple industrial communication protocols and offers both analog and digital input/output capabilities. Its modular architecture enables adaptation to various industrial applications, ranging from environmental monitoring and energy management to process automation and equipment supervision. By centralizing these functionalities in a single platform, the IOBox reduces deployment complexity and accelerates the development of industrial IoT solutions.

The objective of this end-of-study project is to participate in the design and implementation of the IOBox platform. The work focuses on the development of embedded software components, communication interfaces, peripheral drivers, sensor integration, and system architecture required to ensure reliable operation in industrial environments.

This report, which crowns the work entrusted to us, is structured around three chapters:

Chapter 1 presents the general context of the project, the host company, the problem statement, the project objectives, and the adopted development methodology.

Chapter 2 introduces the general concept of the IOBox platform and describes the hardware components, software tools, communication protocols, sensors, and peripherals used throughout the project.

Chapter 3 details the design and implementation of the platform, including the software architecture, peripheral configuration, driver development, communication management, sensor integration, and system validation.

CHAPTER 1

GENERAL PROJECT CONTEXT

1.1 Introduction

This chapter gives an overview of the general setting for the graduation project and the environment where it took place. It starts by introducing the host company, Be Wireless Solutions (BWS), explaining its structure, the kinds of work it does, and the main IoT solutions it offers. The chapter then outlines the project's requirements by explaining the problem that was identified, the idea behind the IO-Box platform, the development method used, and the project's goals and anticipated effects. This summary helps set the stage for understanding the technical information discussed in the following chapters.

1.2 Academic Institution

The Higher Institute of Computer Science and Mathematics at the University of Monastir (ISIMM) was created by decree number 1623 on July 9, 2002. It is a public, scientific higher education institution that is supervised by the Ministry of Higher Education and Scientific Research[2]. ISIMM provides engineering programs in Computer Science, Mathematics, and Electronics. This graduation project was done in the Electronics department, with a focus on Microelectronics, as part of the requirements for the National Diploma in Engineering in Applied Sciences and Technology.

1.3 Host Company Presentation

1.3.1 Company Overview

Be Wireless Solutions (BWS) is a Tunisian engineering company that focuses on IoT systems, embedded electronics, and industrial monitoring. The company creates and makes connected devices, communication gateways, smart sensors, and cloud-based platforms that help with real-time monitoring, control, and optimization of industrial processes. BWS has knowledge in hardware design, embedded software development, wireless communication, and cloud services, allowing it to provide full industrial IoT solutions across various industries. Since it was started, BWS has kept growing its business and formed partnerships both in Tunisia and around the world. The company has become skilled in areas like industrial automation, energy management, smart farming, fleet management, environmental monitoring, and remote equipment oversight.



Figure 1.1: Be Wireless Solutions (BWS) logo

1.3.2 Organization Chart

BWS has different departments that work together during the whole process of creating industrial and IoT solutions. These departments are management, research and development, hardware design, embedded software development, cloud services, technical support, and business development.

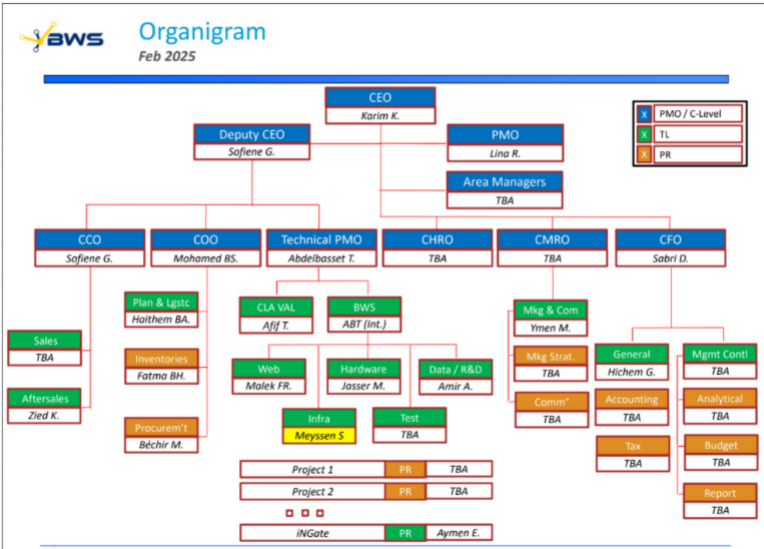


Figure 1.2: Organization chart of Be Wireless Solutions

1.3.3 Main Activities and Services

BWS creates many different industrial IoT solutions that help make operations more efficient, manage resources better, and keep track of processes more effectively.

Energy Monitoring and Management

BWS provides intelligent energy monitoring systems capable of collecting, analyzing, and visualizing electricity consumption data in real time. These solutions help organizations identify energy-intensive equipment, optimize consumption, reduce operational costs, and improve environmental performance.

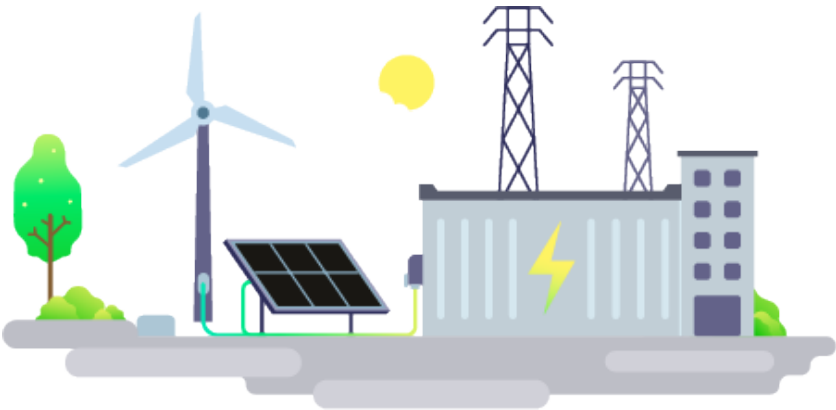


Figure 1.3: Electricity consumption monitoring solution

Water Monitoring Solutions

The company provides smart water management solutions that allow for remote reading of meters, detecting leaks, monitoring unusual water use, and helping to use water resources more efficiently.

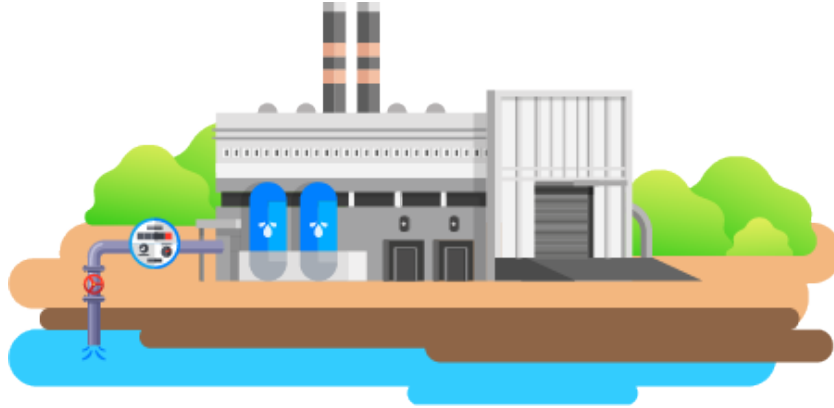


Figure 1.4: Water consumption monitoring solution

Fleet and Fuel Management

BWS develops fleet management systems that provide real-time vehicle tracking, fuel consumption analysis, route optimization, and driver behavior monitoring.



Figure 1.5: Fleet and fuel management solution

Remote Monitoring and Equipment Control

The company provides remote supervision solutions that allow operators to monitor industrial equipment, detect anomalies, generate alerts, and perform remote control operations through dedicated software platforms.

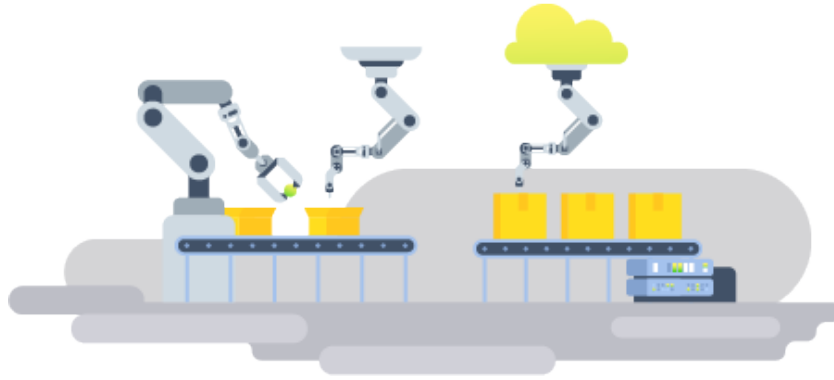


Figure 1.6: Remote equipment monitoring and control

Cold Chain Monitoring

BWS also offers cold-chain monitoring systems intended for warehouses, refrigerated transportation, and pharmaceutical storage facilities. These solutions ensure compliance with temperature requirements and provide instant alerts when abnormal conditions occur.



Figure 1.7: Cold-chain monitoring solution

1.4 Project Specification

1.4.1 Problem Statement

Industrial IoT systems usually use many sensors, actuators, and communication interfaces. Before this project, BWS created custom firmware for each new client setup. This meant they had to reconfigure and rebuild drivers every time there was a new mix of sensors, actuators, and communication interfaces. This method made development

take longer, limited the ability to reuse code between projects, and made it harder to maintain the system when hardware changed. To fix these issues, BWS started the IOBOX project. The goal was to build a flexible, hardware-independent platform that could support various setups using one common firmware base.

In industrial settings, there are often different communication protocols and interface standards. Handling these through custom solutions adds complexity and makes it harder to scale the system.

To tackle these challenges, Be Wireless Solutions began developing the IOBOX platform. The aim is to offer a customizable and reusable system for industrial data collection and control. This system aims to make adding peripherals easier while keeping the system flexible, easy to move between setups, and simple to maintain.

1.4.2 Presentation of the IOBOX Platform

The IOBOX is a flexible industrial system that helps connect sensors, actuators, and communication tools in industrial setups.

It uses a modular design that separates the parts that depend on specific hardware from the parts that handle the main functions. This is done using special software tools like drivers, a Board Support Package, and a Hardware Abstraction Layer, which make it easier to use across different systems and keep it easy to update.

The IOBOX can work with various types of industrial equipment such as analog and digital inputs and outputs, communication ports, and measurement devices. Its software setup can be changed to fit different industrial needs without needing big changes to the firmware.

By linking the hardware to the software in a simple way, the IOBOX makes it easier to put together and use in industrial environments.

1.4.3 Project Objectives

The main goal of this project is to create and build the IOBOX platform, which can collect, handle, control, and send industrial data as it happens. More specifically, the project aims to:

- Create and set up a flexible industrial system for gathering data and managing control.
- Provide a unified software abstraction layer for heterogeneous peripherals.
- Support configurable analog and digital input/output functionalities.

- Implement industrial communication protocols including Modbus RTU over RS485, CAN, LIN, and multi-channel UART, enabling interoperability with standard industrial equipment.
- Develop a modular software architecture based on abstraction layers and reusable software components.
- Design the software architecture to allow new sensors, actuators, and communication interfaces to be added with minimal modification to existing code, through compile-time feature flags and a consistent BSP layer.

The developed platform contributes to reducing development effort and deployment time by providing a standardized and reusable embedded architecture capable of supporting a wide range of industrial applications.

1.4.4 Development Methodology

The IOBOX platform was built using the V-Model, a structured way of developing systems that's commonly used in embedded and industrial projects.

The V-Model connects each step of the development process with a matching testing step. This helps make sure that every part of the system, from the original requirements to the final tests, can be clearly traced back to its source.

In this project, the V-Model was followed closely. During the requirements phase, the platform's features and technical needs were set based on what BWS needed. The system design phase created the four-layer software structure and picked the right hardware parts. The software design phase developed the BSP module interfaces, the communication manager designs, and the sensor driver details. The implementation phase included writing all the BSP drivers, sensor managers, communication modules, and the application layer. Testing was done by checking each BSP module individually, testing the sensor data flow and Modbus communication together, and finally testing the whole system to make sure it works as planned.

Table 1.1: Application of the V-Model to the Project

V-Model Phase	Project Activity
Requirements Analysis	Platform specifications derived from BWS requirements.
System Design	Hardware selection and software architecture definition.
Software Design	BSP API design and manager module interface definition.
Implementation	Development of BSP drivers, sensor managers, Modbus communication, CAN communication, and filtering algorithms.
Unit Testing	Validation of individual BSP modules and drivers.
Integration Testing	Verification of sensor chains and end-to-end Modbus communication.
System Testing	Complete platform validation on the target hardware.

The ascending branch focuses on checking and making sure everything works correctly through unit testing, integration testing, system testing, and final acceptance testing.

The use of the V-Model helps improve the quality of the software, lowers the risks during development, and makes it easier to manage the project all through its different stages.

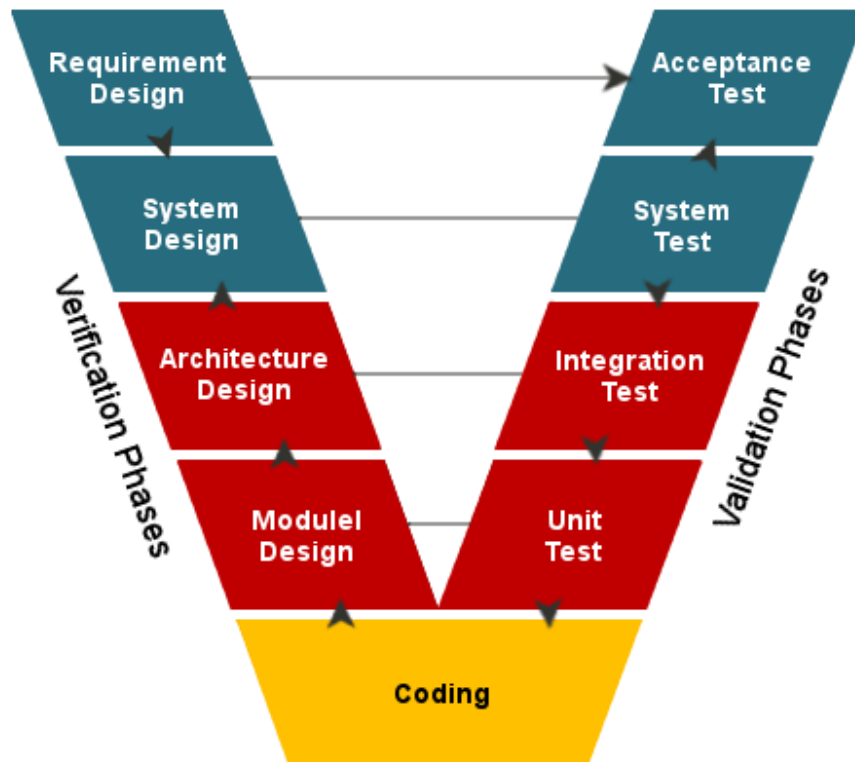


Figure 1.8: V-Model development lifecycle

1.5 Conclusion

This chapter explained the main structure of the project. It started by introducing the university and the company where the project is based. Then, it gave a summary of what the company does and the technologies they use. Next, it went into more detail about the project's requirements, the problem it aims to solve, its goals, and the approach used to develop it.

The next chapter talks about the technical setup of the IOBox platform. It covers the physical parts of the system, the software tools used for development, the ways devices communicate with each other, and the extra parts of the system that support the main functions.

CHAPTER 2

GENERAL CONCEPT AND SYSTEM COMPONENTS

2.1 Introduction

This chapter talks about the technical setup and the main parts used to build the IO-Box platform. It covers the hardware and software that make up the system, like the STM32G474 microcontroller, external peripherals, sensors, tools for development, and communication protocols.

The chapter also explains the main STM32 parts used in the project, such as Analog-to-Digital Converters (ADC), Digital-to-Analog Converters (DAC), timers, and Pulse Width Modulation (PWM) features. Knowing about these technologies is important for understanding the design and how things work, which will be explained in the next chapters.

2.2 General Concept of the IO-Box Platform

The IO-Box is a customizable industrial Input/Output system that helps connect sensors, actuators, and communication tools in IoT and automation setups.

It works as a bridge between field devices and control systems by gathering data from sensors, handling that data, managing external equipment, and sharing information using different communication protocols.

The system is built with a modular design, meaning it can include various hardware parts and software features based on what's needed for each project. This adaptability makes it useful for many industrial tasks like monitoring, automating, and collecting

data.

All these features—like strong embedded processing, multiple communication options, the ability to read both analog and digital signals, and reliable storage—are combined into one easy-to-use device.

2.3 Hardware components

2.3.1 STM32G474MBTx Microcontroller

At the center of the IO-Box hardware design is the STM32G474 microcontroller, a powerful chip made by STMicroelectronics. It uses the ARM Cortex-M4 processor with a floating-point unit, which makes it very good option for handling real-time signal processing and control tasks.

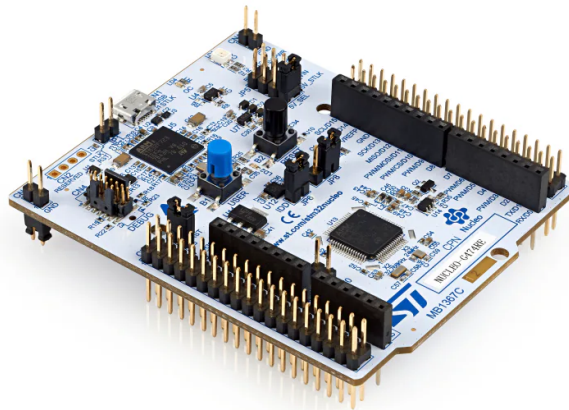


Figure 2.1: STM32G474 Microcontroller (physical package)

The STM32G474 was chosen instead of the STM32F4 series because it has more ADC units (five compared to two on the F4), includes a built-in FDCAN feature, and uses a newer HAL environment. When compared to the STM32L4 series, the G474 has a faster clock speed (170 MHz versus 80 MHz) and a better analog front end, which are important for handling real-time signal processing at the needed sampling rates.

Table 2.1: Product Attributes — STM32G474 Microcontroller

Category	Integrated Circuits, Embedded Systems, Microcontrollers
Manufacturer	STMicroelectronics
Series	STM32G4
Processor Core	ARM Cortex-M4F
Core Size	32-Bit
Speed	170 MHz
Number of I/O	52
Program Memory Size	512 KB Flash
RAM Size	128 KB
Data Converters	ADC: 26×12 -bit; DAC: 7×12 -bit
Supply Voltage	1.71 V – 3.6 V
Oscillator Type	Internal
Operating Temperature	$-40\text{ }^{\circ}\text{C}$ to $+125\text{ }^{\circ}\text{C}$
Mounting Type	Surface Mount
Package	64-LQFP (10×10 mm)
Base Product Number	STM32G474

Inside the IO-Box, the STM32G474 serves as the main processor. It handles tasks like collecting signals, processing data locally, and controlling actuators. Additionally, it manages communication with other systems using various industrial protocols, which is essential for achieving the project’s goals.

2.3.2 EEPROM Memory: M95M01-RDW6TP

To manage the work with the STM32 microcontroller and make sure data is stored reliably, the IO-Box uses an external EEPROM memory called the M95M01-RDW6TP from STMicroelectronics. This device stores important information like settings, calibration data, and system logs, so the system can keep this data even when the power is turned off.

The M95M01-RDW6TP is a 1-Mbit serial EEPROM that is organized into 131,072 memory locations, each holding 8 bits. It has a fast response time of 25 nanoseconds and connects through the SPI bus, which makes it easy to use with the STM32G474. It comes in an 8-pin SOIC package, making it a small and efficient option for embedded systems. It can handle up to one million write operations and keeps data safely for up to 40 years, making it a dependable choice for industrial uses where reliability is key.

Table 2.2: Key Characteristics of M95M01-RDW6TP EEPROM

Feature	Specification
Manufacturer	STMicroelectronics
Type	Serial EEPROM
Capacity	1 Mbit ($131,072 \times 8$ bits)
Interface	SPI
Access Time	25 ns
Package	SOIC-8
Endurance	1,000,000 write cycles
Data Retention	40 years
Operating Voltage	1.8 V – 5.5 V

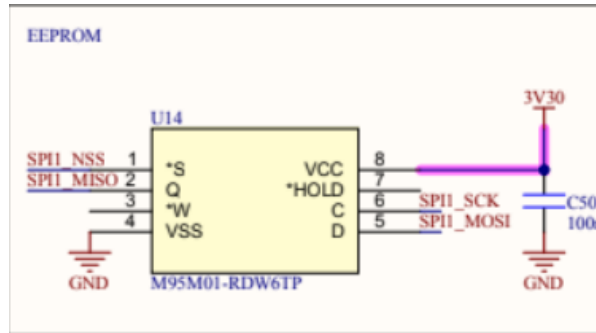


Figure 2.2: M95M01-RDW6TP EEPROM memory chip

2.3.3 Sensors

In the IOBox project, we added several sensors to measure different environmental and physical conditions. Our main task was to connect these sensors to the STM32G474

microcontroller, set up their settings, handle how they communicate, and do some testing to make sure the data they collect is correct. We looked closely at each sensor, learned about how they work, and wrote down their features to show how each one contributes to the whole system.

ENS210 Sensor

The ENS210 is a digital sensor made by ScioSense that measures both temperature and humidity in a small size. It uses an I²C interface to connect easily with microcontrollers like the STM32G474. The sensor is known for being accurate and using very little power, which makes it a good choice for use in systems that monitor the environment.

Table 2.3: Key Characteristics of ENS210 Sensor

Feature	Specification
Manufacturer	ScioSense
Type	Temperature + Humidity Sensor
Interface	I ² C (16-bit output registers)
Supply Voltage	1.71 V – 3.6 V
Temperature Range	–40 °C to +100 °C
Temperature Accuracy	±0.15 °C
Humidity Range	0 – 100 % RH
Humidity Accuracy	±2 % RH
Package	4-QFN (2 × 2 mm)

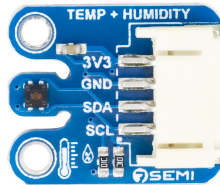


Figure 2.3: ENS210 digital temperature and humidity sensor

OPT3001DNPR Sensor

The OPT3001DNPR is a digital light sensor made by Texas Instruments. It measures how bright the light is and shows the results in units called lux. This makes it great for uses like automatically adjusting screen brightness, controlling display backlights, and monitoring the environment. The sensor uses the I²C communication method and has built-in calibration and a linear response over a wide range of light levels.

In the IO-Box project, the OPT3001DNPR was used to get precise light readings. Work was done on its internal settings to set up different measurement modes, set thresholds, and run calibration processes, ensuring the sensor gave dependable data for real-time use.

Table 2.4: Key Characteristics of OPT3001DNPR Sensor

Feature	Specification
Manufacturer	Texas Instruments
Type	Ambient Light Sensor
Interface	I ² C
Supply Voltage	1.6 V – 3.6 V
Measurement Range	0.01 lux – 83,000 lux
Output Format	16-bit digital result in lux
Accuracy	±10% typical
Operating Temperature	−40 °C to +85 °C
Package	6-Pin WSON (2 × 2 mm)



Figure 2.4: OPT3001DNPR ambient light sensor

HTS221TR Sensor

The HTS221TR is a capacitive digital humidity and temperature sensor developed by STMicroelectronics. It integrates sensing elements and signal conditioning circuitry within a compact package, providing calibrated measurements through a standard I²C interface. The sensor is designed for low-power applications while maintaining good measurement accuracy, making it suitable for environmental monitoring and industrial IoT systems.

Within the IO-Box platform, the HTS221TR is used to acquire ambient temperature and relative humidity data. Its factory-calibrated coefficients simplify integration and improve measurement reliability. Communication with the STM32G474 microcontroller is achieved through the I²C bus, and dedicated software drivers were developed to configure the device and process sensor data.

Table 2.5: Key Characteristics of HTS221TR Sensor

Feature	Specification
Manufacturer	STMicroelectronics
Type	Temperature + Humidity Sensor
Interface	I ² C (SPI mode not used in this project)
Supply Voltage	1.7 V – 3.6 V
Temperature Range	-40 °C to +120 °C
Temperature Accuracy	±0.5 °C (typ.)
Humidity Range	0 – 100 % RH
Humidity Accuracy	±3.5 % RH (typ.)
Output Resolution	16-bit
Package	HLGA-6 (2 × 2 mm)

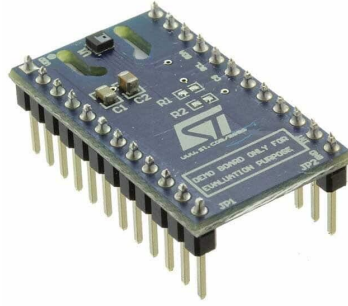


Figure 2.5: HTS221TR temperature and humidity sensor

2.4 Peripherals

2.4.1 Digital-to-Analog Converter (DAC)

The Digital-to-Analog Converter (DAC) is an analog output peripheral integrated within the STM32G474MBTx microcontroller. It converts digital values stored in memory into continuous analog voltage levels with a resolution defined by the DAC bit width. This allows the generation of precise voltage references or analog waveforms such as sine waves, ramps, or arbitrary signals.

The DAC can operate in several modes, including software-triggered mode, timer-triggered mode, and DMA-driven mode. In timer-triggered mode, a hardware timer periodically updates the DAC output, ensuring precise and deterministic signal timing. When combined with Direct Memory Access (DMA), the DAC can continuously output precomputed waveform data without CPU intervention, significantly improving efficiency.

In the IOBox platform, the DAC is used to generate analog reference signals for actuator control. Two DAC channels are available, supporting both software-triggered and timer-triggered output modes.

An example of DAC signal generation is illustrated in Figure 2.6.

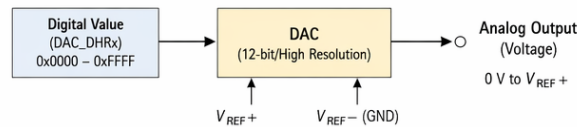


Figure 2.6: Example of analog signal generated by DAC

2.4.2 Analog to Digital Converter (ADC)

The Analog-to-Digital Converter (ADC) is a fundamental peripheral in embedded systems, responsible for transforming continuous analog signals into discrete digital values that can be processed by the microcontroller. This conversion is essential for applications where physical phenomena such as temperature, pressure, or current must be measured and interpreted by digital logic.

In the IOBox project, ADCs are employed to acquire signals from sensors and external modules, enabling precise monitoring and control. The STM32G474 microcontroller integrates multiple high-resolution ADC channels, allowing simultaneous sampling of different inputs. This capability is particularly important in industrial contexts where multiple signals must be captured in real time.

The operation of an ADC follows a well-defined process. The analog input is sampled at regular intervals determined by the sampling frequency. Each sample is then quantized into a digital value according to the converter's resolution (e.g., 12-bit or 16-bit). The resulting digital word represents the amplitude of the analog signal at that instant. The accuracy of this conversion depends on factors such as resolution, sampling rate, and reference voltage stability.

Key characteristics of ADCs include:

- **Resolution:** Defines the number of discrete levels available.
- **Sampling Rate:** Determines how frequently the analog signal is measured, affecting the fidelity of dynamic signals.
- **Reference Voltage:** Sets the maximum measurable input range, ensuring proper scaling of the digital output.
- **Input Channels:** Multiple channels allow acquisition from several sensors simultaneously.
- **Conversion Modes:** Support for single, continuous, and DMA-driven conversions, enabling flexible integration into firmware.

Within the IO-Box, ADCs are used to capture sensor data such as voltage levels, current measurements, or other analog signals. These values are then processed by the firmware to generate meaningful information, trigger control actions, or communicate results to external supervisory systems.

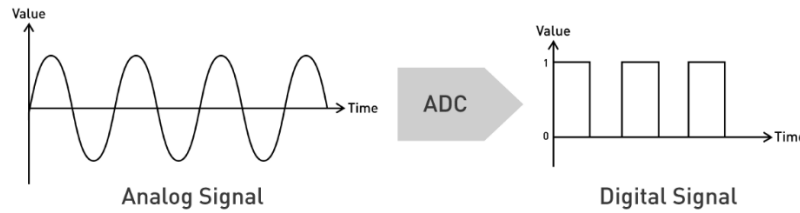


Figure 2.7: Basic ADC conversion from analog input to digital output

2.4.3 Timers

Timers are fundamental peripherals in the STM32G474 microcontroller, designed to provide accurate time base generation, event counting, and signal measurement. They operate by incrementing or decrementing a counter register according to a clock source, and can trigger actions when predefined values are reached.

Key functionalities of timers include:

- **Time Base Generation:** Creating precise delays or periodic events.
- **Input Capture:** Measuring external signal characteristics such as frequency or pulse width.
- **Output Compare:** Generating events when the counter matches a reference value.
- **Synchronization:** Coordinating multiple timers for complex control tasks.
- **Advanced Features:** Dead-time insertion, break inputs, and complementary outputs for safe power electronics control.

Timers are essential for scheduling tasks, measuring sensor signals, and driving peripherals that require precise timing.

2.4.4 Pulse Width Modulation (PWM)

Pulse Width Modulation (PWM) is a technique used to encode analog information into a digital waveform by varying the duty cycle of a periodic signal. In embedded systems, PWM is widely used for motor control, LED dimming, and power regulation.

PWM signals in the STM32G474 are generated using timers. The timer counts up to a predefined value stored in the auto-reload register (ARR), while the compare register (CCR) defines the switching point of the output. The ratio between the high

time and the total period represents the duty cycle. By adjusting ARR and CCR, both frequency and duty cycle can be controlled with high precision.

Key characteristics of PWM include:

- **Frequency:** Determined by prescaler and ARR configuration.
- **Duty Cycle:** Defined by CCR relative to ARR.
- **Resolution:** Limited by timer bit width (typically 16-bit).
- **Complementary Outputs:** Enable safe control of MOSFET drivers in half-bridge or full-bridge circuits.
- **Applications:** Motor drivers, LED dimming, switching converters, and DAC emulation.

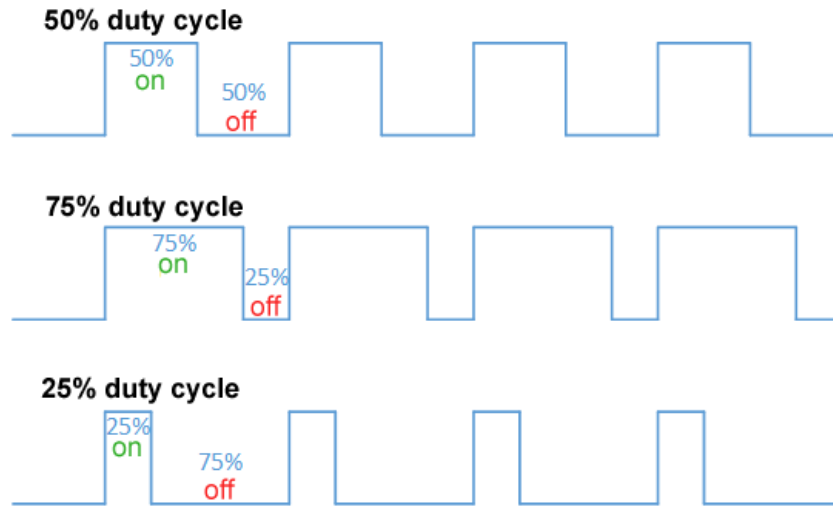


Figure 2.8: Illustration of PWM signals with different duty cycles (25%, 50%, and 75%)

2.5 Communication Protocols

2.5.1 I2C Protocol

Inter-Integrated Circuit (I2C) is a synchronous, half-duplex serial communication protocol widely used for communication between integrated circuits over short distances. It relies on two bidirectional lines: the Serial Data Line (SDA) and the Serial Clock Line (SCL), both requiring pull-up resistors. The protocol follows a master-slave architecture, where the master initiates communication and generates the clock signal.

Each device connected to the bus is identified by a unique address. Data is

then exchanged one byte at a time, with each byte followed by an acknowledgment bit (ACK/NACK) from the receiving device. The communication is terminated by a STOP condition. I2C supports different speed modes, including Standard Mode (100 kHz), Fast Mode (400 kHz), and higher-speed variants.

The I2C frame structure is illustrated in Figure 2.9.

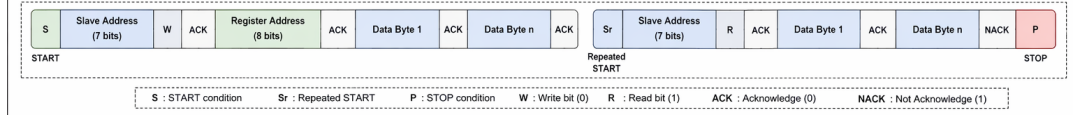


Figure 2.9: I2C frame format

In this project, the I2C interface of the STM32G474MBTx is configured using the HAL library generated by STM32CubeMX. It is used to communicate with external peripherals such as sensors and low-speed devices. The implementation relies on interrupt or polling-based data transfers depending on timing constraints. Particular attention is given to bus stability, including proper pull-up resistor sizing and handling of potential communication errors such as bus busy states or missing acknowledgments.

As shown in Figure 2.10, the I2C bus allows multiple slave devices.

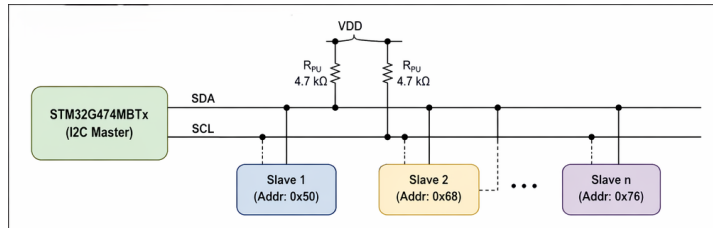


Figure 2.10: I2C communication bus architecture

2.5.2 Serial Peripheral Interface (SPI)

The Serial Peripheral Interface (SPI) is a synchronous serial communication protocol widely used in embedded systems. It enables high-speed data exchange between a microcontroller and peripheral devices such as sensors, memory modules, or communication transceivers. SPI operates in a master-slave configuration, where the microcontroller typically acts as the master, controlling the clock and data flow.

In the IOBOX project, SPI is employed to ensure reliable and efficient communication with external modules that require fast data transfer. Its simplicity and speed make it suitable for real-time applications, particularly when multiple peripherals must be interfaced with the STM32 microcontroller.

SPI communication is based on a clock signal generated by the master device. Each bit of data is shifted out on the MOSI line while simultaneously another bit is shifted in on the MISO line. This process is synchronized by the clock (SCLK), ensuring deterministic timing. The communication sequence typically follows these steps: the master activates the Chip Select (CS) line of the desired slave, provides the clock signal, transmits data bit-by-bit on MOSI while receiving data on MISO, samples the data according to the configured mode, and finally deactivates the CS line to end the session. This mechanism allows full-duplex communication, meaning data can be transmitted and received simultaneously.

SPI supports four modes of operation (Mode 0 to Mode 3), determined by the configuration of clock polarity (CPOL) and clock phase (CPHA). This flexibility ensures compatibility with a wide range of peripheral devices. Key characteristics of SPI include its high speed (operating at several MHz), simple wiring with four main signals (MOSI, MISO, SCLK, CS), and scalability through multiple slave connections.

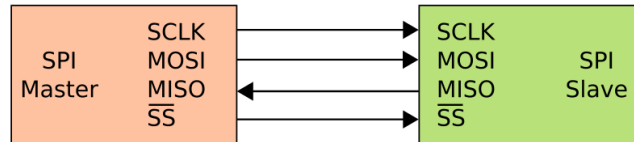


Figure 2.11: Basic SPI communication between master and slave devices

2.5.3 UART Protocol

Universal Asynchronous Receiver-Transmitter (UART) is an asynchronous serial communication protocol that enables full-duplex data exchange between devices without a shared clock signal. Data is transmitted over two lines: Transmit (TX) and Receive (RX). Synchronization between devices is achieved by configuring identical communication parameters, including baud rate, data bits, parity, and stop bits.

Each data frame typically consists of a start bit, followed by a configurable number of data bits (usually 8), an optional parity bit for error detection, and one or more stop bits. Unlike synchronous protocols, UART does not include a clock line, making it simpler but more sensitive to timing mismatches.

In this project, UART is used for communication between the STM32 microcontroller and external systems such as a host computer or other embedded modules. It plays a key role in debugging, logging, and real-time data exchange. The implementation uses the STM32 HAL drivers with support for interrupt-driven and DMA-based

transmission to ensure efficient and non-blocking communication. Error handling mechanisms, including overrun, framing, and noise errors, are also considered to improve communication reliability.

The UART frame structure is illustrated in Figure 2.12.

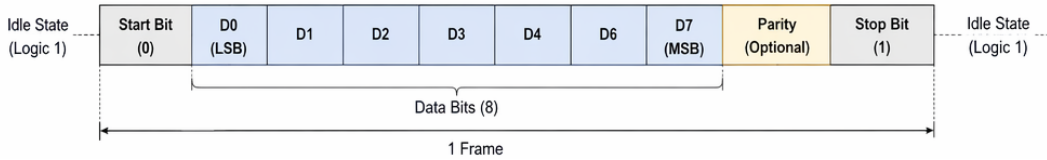


Figure 2.12: UART frame format

2.5.4 RS485 Communication

RS485 is a robust differential serial communication standard widely used in industrial environments due to its high noise immunity and long-distance capabilities. Unlike single-ended UART communication, RS485 transmits data over a differential pair of lines (A and B), where the information is encoded as a voltage difference between the two conductors. This approach significantly reduces susceptibility to electromagnetic interference and ground potential differences.

The RS485 standard supports half-duplex communication in multi-drop configurations, allowing multiple nodes to share the same bus. However, only one device can transmit at a time, which requires proper bus arbitration at the software level or through protocol design.

In this project, RS485 serves as the physical layer for Modbus RTU communication. A dedicated UART peripheral (USART3) is used with an external RS485 transceiver to convert TTL logic levels to differential signals. Direction control is managed in software, as described in Chapter 3.

Before transmission, the microcontroller sets the DE pin high to enable the driver stage. After sending the data frame, the DE pin is reset to return the transceiver to reception mode. This timing control is critical to avoid bus contention. The communication is typically used for reliable data exchange in noisy environments, especially when longer cable lengths are required compared to standard UART.

Figure 2.13 illustrates a typical RS485 bus configuration.

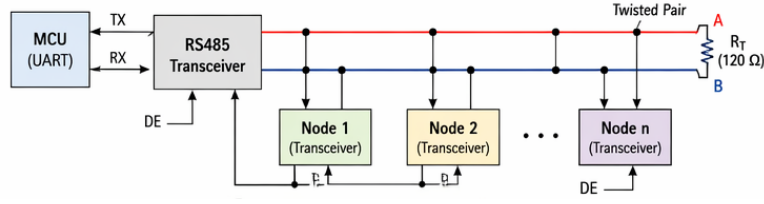


Figure 2.13: RS485 differential bus with multi-node configuration

2.5.5 Modbus Protocol

Modbus is one of the most widely used industrial communication protocols for exchanging data between electronic devices. Originally developed by Modicon in 1979, it follows a client-server (master-slave) architecture and enables communication between programmable logic controllers (PLCs), sensors, actuators, Human-Machine Interfaces (HMIs), and embedded systems.

Modbus defines a standardized message structure that allows devices from different manufacturers to communicate using a common protocol. Two main variants are commonly used: Modbus RTU, which operates over serial communication interfaces such as RS232 and RS485, and Modbus TCP, which operates over Ethernet networks.

In the IOBOX project, Modbus RTU is implemented over the RS485 physical layer to ensure reliable communication in industrial environments. The STM32G474 microcontroller acts as either a Modbus master or slave depending on the application requirements. Through Modbus registers, external systems can access sensor measurements, configuration parameters, status information, and control commands.

In this project, the IOBox operates as a Modbus RTU slave at address 0x01, communicating over RS485 at 19,200 bps. The slave implementation supports function codes FC01 through FC06, FC15, and FC16. Input registers expose real-time sensor measurements (temperature, humidity, illuminance) to any connected Modbus master, and holding registers allow actuator control. The implementation details are described in Chapter 3.

A Modbus RTU frame consists of four main fields:

- **Slave Address:** Identifies the target device.
- **Function Code:** Specifies the requested operation.
- **Data Field:** Contains transmitted data or register information.
- **CRC Field:** Provides error detection for frame integrity.

The protocol offers several advantages for industrial applications:

- Simplicity of implementation.
- Wide industrial adoption.
- Interoperability between devices from different vendors.
- Reliable communication over long distances using RS485.
- Easy integration with supervisory and SCADA systems.

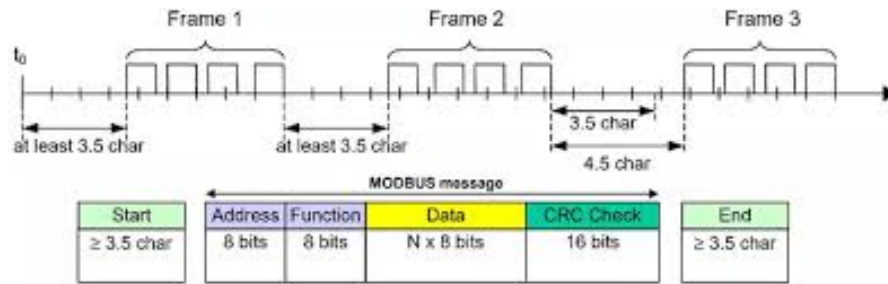


Figure 2.14: Typical Modbus RTU frame structure

2.5.6 Controller Area Network (CAN)

The Controller Area Network (CAN) is a robust serial communication protocol originally developed for automotive applications, but now widely used in industrial and embedded systems. It enables reliable data exchange between multiple devices without requiring a central host, making it ideal for distributed control systems.

In the IOBOX project, CAN is employed to ensure communication with industrial equipment and sensors that demand high reliability and fault tolerance. Its ability to handle multiple nodes on a single bus makes it particularly suitable for environments where numerous devices must share information in real time.

CAN communication is based on a message-oriented protocol. Instead of addressing specific devices directly, each message carries an identifier that defines its priority and content. This allows multiple nodes to listen simultaneously and decide whether to act on the received data. The arbitration mechanism ensures that when two nodes attempt to transmit at the same time, the message with the highest priority (lowest identifier value) wins, while the other node waits until the bus is free. This guarantees deterministic communication without collisions.

Key characteristics of CAN include:

- **Multi-master capability:** Any node can initiate communication when the bus

is idle.

- **Error detection and handling:** Built-in mechanisms such as CRC checks, acknowledgment bits, and error frames ensure data integrity.
- **Priority-based arbitration:** Messages are prioritized by identifiers, ensuring that critical data is transmitted first.
- **Scalability:** Supports up to 112 nodes on a single bus, making it suitable for complex systems.
- **Speed:** Operates up to 1 Mbps in classical CAN, and higher in CAN FD (Flexible Data-Rate), which extends payload size and transmission speed.

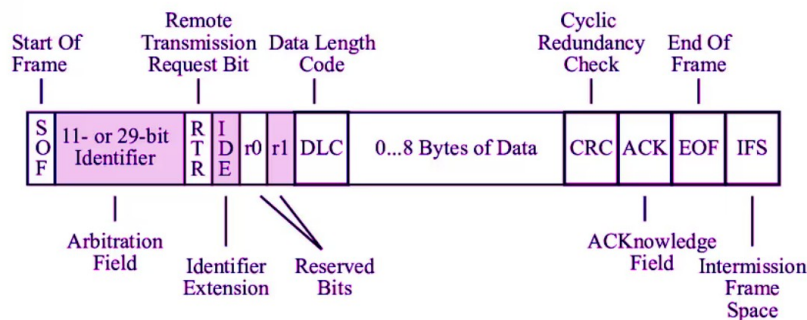


Figure 2.15: Basic CAN frame

In practice, CAN uses two differential lines, CAN_H and CAN_L, to transmit data. This differential signaling improves noise immunity and allows reliable communication even in electrically harsh environments. Each frame consists of fields such as the start of frame, identifier, control bits, data payload, CRC, and end of frame, ensuring structured and error-resistant communication.

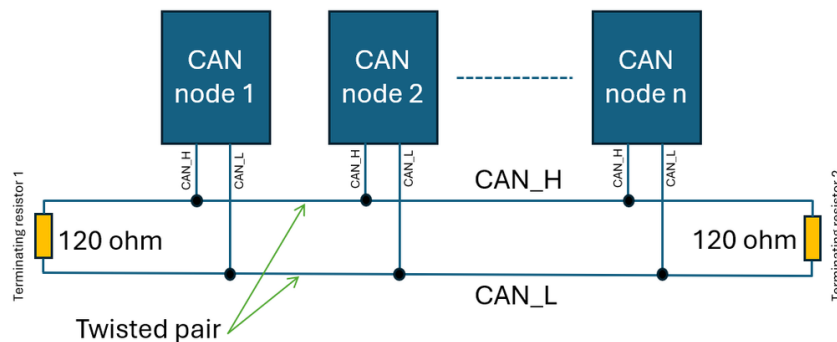


Figure 2.16: Basic CAN bus communication with multiple nodes

2.5.7 LIN Protocol

Local Interconnect Network (LIN) is a low-cost serial communication protocol designed for distributed control systems, particularly in automotive applications. It operates on a single-wire physical layer and follows a strict master-slave architecture where the master node controls the entire bus communication schedule.

Unlike event-driven protocols, LIN is time-triggered, meaning communication is organized into predefined frames sent at fixed intervals. Each frame consists of a header transmitted by the master and a response transmitted by a slave node. The header includes a break field (used to signal the start of a frame), a synchronization byte, and an identifier field that determines which node should respond.

The data response typically contains up to 8 bytes, followed by a checksum used for error detection. LIN achieves deterministic timing by enforcing strict scheduling, making it suitable for non-critical but predictable control tasks.

In this project, LIN communication is implemented using the STM32 UART peripheral configured in LIN mode. The hardware support simplifies break detection and synchronization field handling, reducing software complexity. The system can therefore reliably detect frame boundaries and process incoming data with minimal CPU overhead.

The LIN frame structure is shown in Figure 2.17.

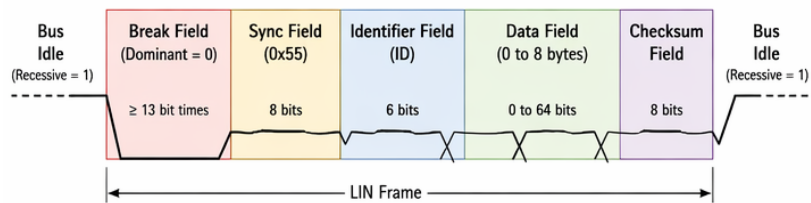


Figure 2.17: LIN frame structure

2.6 Software tools

2.6.1 Visual Studio Code

Visual Studio Code (VS Code) is a lightweight yet powerful source code editor optimized for modern software development. It supports multiple programming languages and provides advanced features such as syntax highlighting, intelligent code completion, and debugging tools.

In this project, Visual Studio Code was used as the primary text editor for documentation, script editing, and Git-based version control. Extensions such as Git Graph

enabled visualization of commit history, branch management, and tracking of code evolution across the development team.



Figure 2.18: Visual Studio Code interface

2.6.2 IAR Embedded Workbench

IAR Embedded Workbench (IAR EW) is a professional integrated development environment (IDE) dedicated to embedded systems programming, particularly for microcontrollers such as the STM32 family. It provides a highly optimized C/C++ compiler, advanced debugging capabilities, and efficient project management tools.

In this project, IAR EW was used for firmware development, compilation, and debugging. Its powerful optimization features contributed to generating efficient and reliable embedded code, which is critical for real-time applications and resource-constrained systems[1].



Figure 2.19: IAR Embedded Workbench IDE

2.6.3 STM32CubeMX

STM32CubeMX is a graphical configuration tool developed by STMicroelectronics for STM32 microcontrollers. It allows developers to easily configure peripherals, clock settings, and pin assignments through an intuitive interface. The tool also generates initialization code based on the selected configuration, significantly reducing development time and minimizing configuration errors.

In this project, STM32CubeMX was used in a limited capacity: it was invoked once at the beginning of the project to generate the initial project skeleton, configure the system clock (HSI PLL at 170 MHz), and enable the Serial Wire Debug interface. The peripheral initialization functions that CubeMX would normally generate were intentionally not used. Instead, all peripheral drivers were developed as handwritten BSP (Board Support Package) modules, which are described in detail in Chapter 3. CubeMX thus served exclusively as a project scaffolding and clock configuration tool.



Figure 2.20: STM32CubeMX configuration interface

2.6.4 GitLab

GitLab was used as the version control and collaboration platform throughout the project. The repository was organized by module (BSP, Managers, Application), with branches used to isolate feature development and avoid conflicts during parallel work. Commit history served as a development log, and merge requests were used to review code before integrating changes into the main branch.



Figure 2.21: GitLab logo

2.6.5 Conclusion

In this chapter, we presented the general concept and system components of the IO-Box project. We began with the synoptic diagram, which outlined the main functional phases of acquisition, filtering, regulation, and transmission/actuation. We then

introduced the software architecture, organized into four distinct layers (HAL, BSP, Manager, and Application), and complemented this with the hardware architecture, highlighting the STM32G474 microcontroller, power supply design, communication interfaces, and peripheral modules. Finally, we described the development tools and protocols that support the implementation of the system.

By establishing this technical foundation, Chapter 3 has clarified the resources and methodologies that will be employed in the realization phase. The next chapter will be dedicated to the practical implementation of the IO-Box, detailing the firmware design, register configuration, BSP development, and integration of sensors and actuators into a coherent and functional platform.

CHAPTER 3

DESIGN AND PRACTICAL IMPLEMENTATION

3.1 Introduction

3.2 Hardware architecture

3.3 Software architecture

3.3.1 Board support package (BSP) layer

3.3.2 Manager layer

3.3.3 Application layer

3.4 Conclusion

BIBLIOGRAPHY

- [1] IAR EW. consulted on 04/13/2026. <https://www.iar.com/>.
- [2] ISIMM. consulted on 04/13/2026. <http://www.isimm.rnu.tn/public/>.